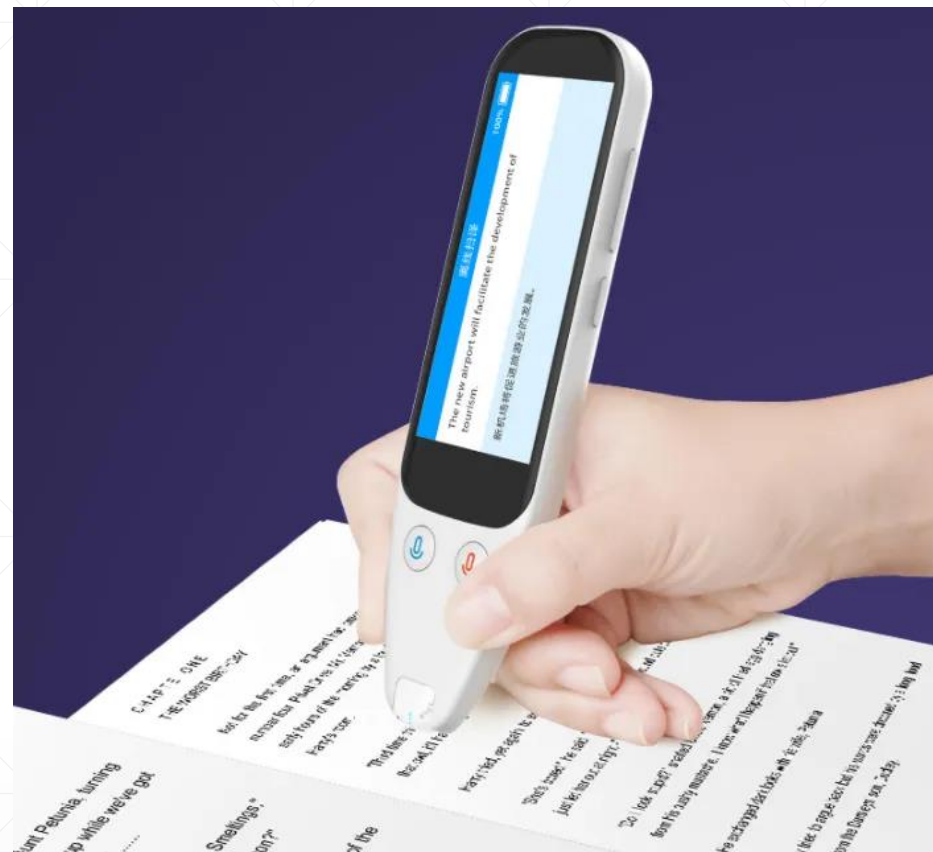


Scanner的用法

北京理工大学计算机学院
金旭亮



概述



Scanner是一个字符流处理组件，它使用特定的分隔符（称为Delimiter，比如空格或逗号），将一个字符流分割为多个“Token”。

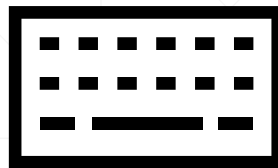


Scanner最常见的用法就是注入一个System.in对象，通过它接收用户的键盘输入。



与其它Java IO组件一样，Scanner在用完之后，必须关闭。

最简单的用法



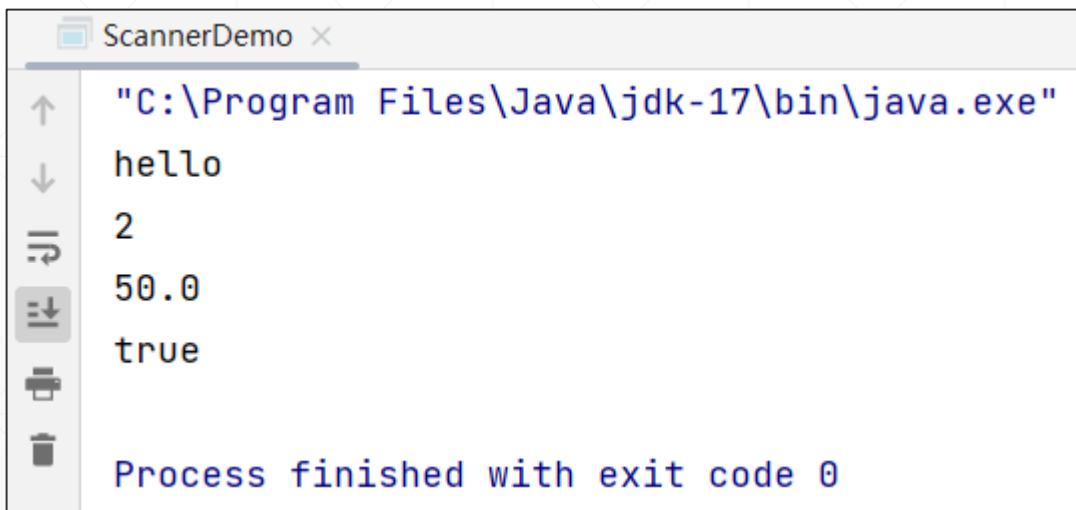
在控制台窗口中接收用户输入（按行读取）：

```
private static void userInputFromKeyboard() {  
    System.out.print("请输入一个字符串: ");  
    Scanner scanner = new Scanner(System.in);  
    String userInput = scanner.nextLine();  
    System.out.println("\n用户输入了: " + userInput);  
}
```

```
ScannerDemo x  
"C:\Program Files\Java\jdk-17\bin\java.exe"  
请输入一个字符串: 好好学习, 天天向上。  
  
用户输入了: 好好学习, 天天向上。  
  
Process finished with exit code 0
```

切分字符串

Scanner使用分隔符（默认是空格）来切分字符串，并且提供了nextXXX()系列方法，读入特定类型的数值。



```
ScannerDemo x
"C:\Program Files\Java\jdk-17\bin\java.exe"
hello
2
50.0
true
Process finished with exit code 0
```

```
//使用next系列方法从字符串中分解Token
private static void scanString() {
    //要处理的原始字符串
    String line = "hello 1 2.5 true";
    var scLine = new Scanner(line);
    //读取第一个Token
    String word = scLine.next();
    System.out.println(word);
    //读取整数
    int num = scLine.nextInt();
    System.out.println(num * 2);
    //读取double数值
    double dNum = scLine.nextDouble();
    System.out.println(dNum * 100 / 5.0);
    //读取boolean数值
    boolean bool = scLine.nextBoolean();
    System.out.println(bool == true);
    scLine.close();
}
```

指定要使用的分隔符

//展示如何指定要切割字符串所使用的分隔符

```
private static void useDelimiter() {  
    String dataString = "abc#1#2.5#true";  
    //指定分隔符  
    Scanner scanner = new Scanner(dataString).useDelimiter("#");  
    var word :String = scanner.next();  
    var num :int = scanner.nextInt();  
    var fnum :float = scanner.nextFloat();  
    var bool :boolean = scanner.nextBoolean();  
    System.out.printf("%s %d %f %b", word, num, fnum, bool);  
    scanner.close();  
}
```

ScannerDemo x

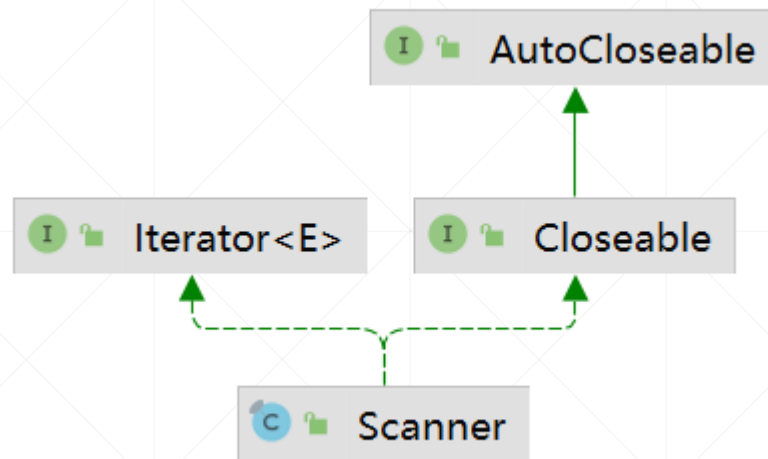
"C:\Program Files\Java\jdk-17\bin\java.exe"

abc 1 2.500000 true

Process finished with exit code 0

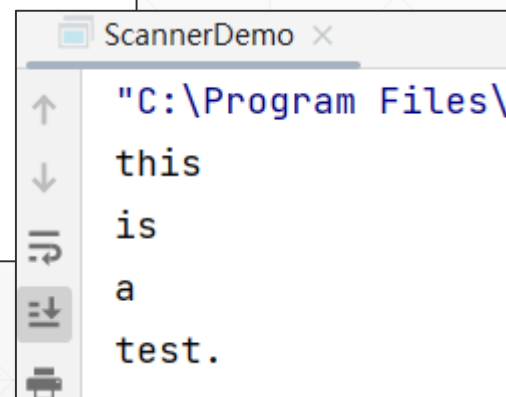
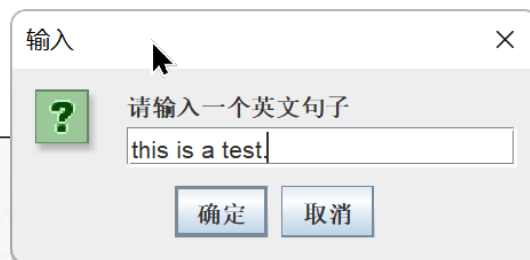
自动关闭Scanner

Scanner实现了Closeable接口，可以用于try表达式中，当流程离开try语句块时，会被自动关闭：



//循环读取和自动关闭

```
private static void loopScanAndAutoClose() {
    var userInput : String = JOptionPane.showInputDialog("请输入一个英文句子");
    try (Scanner scanner = new Scanner(userInput)) {
        while (scanner.hasNext()) {
            System.out.println(scanner.next());
        }
    }
}
```



Scanner可以直接读取文件

scanner的构造函数可以接收InputStream，因此，可以使用它来直接处理文本文件内容：

```
public Scanner(InputStream source)
```

```
public Scanner(InputStream source, String charsetName)
```

```
//读取Java文件中的所有行
private static void readJavaFile() throws FileNotFoundException {
    try (var scanner = new Scanner(new File(pathname: "src/ScannerDemo.java"))) {
        while (scanner.hasNext()) {
            System.out.println(scanner.nextLine());
        }
    }
}
```

应用实例：处理数据文件

```
//从数据文件中逐行读入，并且处理它们
private static void readAndProcessDataFile() throws FileNotFoundException {
    //本地内部类，封装行的处理功能
    class LineProcessor {
        String word, line;
        int num;
        boolean bool;
        void process(Scanner lineScanner) {
            word = lineScanner.next();
            num = lineScanner.nextInt();
            bool = lineScanner.nextBoolean();
            System.out.printf("%s %d %b \n", word, num, bool);
        }
    }
    LineProcessor processor = new LineProcessor();
    String line;
    try (var scanner = new Scanner(new File("data/mydata.txt"))) {
        while (scanner.hasNext()) {
            line = scanner.nextLine();
            try (var lineScanner : Scanner = new Scanner(line).useDelimiter("#")) {
                processor.process(lineScanner);
            }
        }
    }
}
```

使用两个Scanner，一个负责读取行，另一个负责分割行。

mydata.txt	
1	a#1#true
2	b#2#false
3	c#3#true

ScannerDemo

```
"C:\Program Files
a 1 true
b 2 false
c 3 true

Process finished
```